Επίλυση Προβλημάτων

Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος
  - Αναζήτηση πρώτα σε πλάτος (BFS)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων
- Ικανοποίηση Περιορισμών

Προγραμματισμός με Περιορισμούς (CP=Constraint Programming)

- Ο προγραμματισμός με περιορισμούς (CP=Constraint Programming) είναι μια τεχνική αντιμετώπισης **προβλημάτων αναζήτησης** για τα οποία:
  - Οι καταστάσεις αποτελούνται από **«μερικές αναθέσεις»** δηλαδή σε κάποιες μεταβλητές έχουν ανατεθεί τιμές ενώ σε κάποιες άλλες όχι ακόμα.
  - Ο έλεγχος για το εάν έχει παραχθεί λύση καθορίζεται από ένα σύνολο περιορισμών που θα πρέπει να ισχύουν.

Παραδείγματα προβλημάτων CP

- Προβλήματα ανάθεσης
  - Π.χ. ποιος εργαζόμενος θα αναλάβει ποια εργασία.
- Προβλήματα χρονοπρογραμματισμού
  - Π.χ. πότε, που και από ποιον θα διδάσκεται ένα μάθημα.
- Προγραμματισμός μεταφορών
  - Π.χ. ποια μεταφορά θα αναλάβει το κάθε φορτηγό προκειμένου να επιτευχθεί ένα έργο μεταφοράς.
- Προγραμματισμός εργασιών σε βιομηχανίες
  - Π.χ. με ποια σειρά θα γίνουν κάποιες εργασίες σε μια βιομηχανία.

Ικανοποίηση περιορισμών *(constraint satisfaction problem)*

Προβλήματα που αναπαριστώνται από:

- Ένα σύνολο *n* **μεταβλητών** V_1, V_2, V_3, … V_n,
- Ένα σύνολο *n* **πεδίων τιμών** D_1, …D_n, που αντιστοιχούν σε κάθε μεταβλητή έτσι ώστε $V_1 \in D_1$, … $V_n \in D_n$ και γενικά $V_i \in D_i$, και
- Ένα σύνολο σχέσεων (**περιορισμών**) C_1, C_2,…C_m όπου $C_i(V_k,…,V_m)$ μια σχέση μεταξύ των μεταβλητών του προβλήματος π.χ. $V_1 < V_2$, $V_2 \neq V_3$..

Λύση Προβλήματος Περιορισμών

Λύση αποτελεί μια ανάθεση τιμών στις μεταβλητές του προβλήματος, τέτοια ώστε να ικανοποιούνται οι περιορισμοί, δηλαδή:

- *Είδη τιμών των μεταβλητών:*

$V_1 = d_1, V_2 = d_2, \ldots V_n = d_n$

- *Περιορισμοί που αφορούν τα πεδία των μεταβλητών:*

$\wedge\ d \in D_1 \wedge d_2 \in D_2 \wedge \ldots \wedge d_i \in D_n$

- *Περιορισμοί του προβλήματος:*

$\wedge C_1 \wedge C_2 \wedge \ldots \wedge C_m$

Ικανοποίηση περιορισμών

Είδη προβλημάτων:

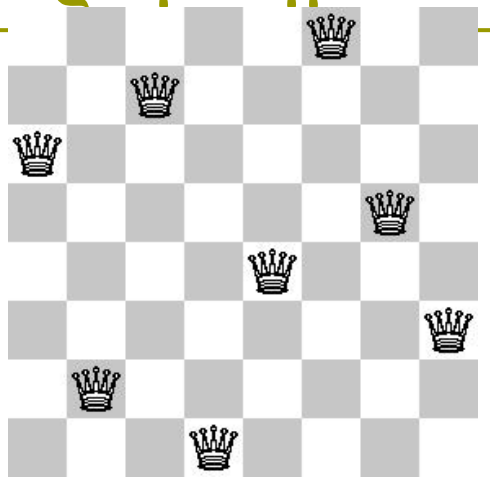
- **Προβλήματα ικανοποίησης περιορισμών** όταν τα πεδία τιμών των μεταβλητών παίρνουν διακριτές τιμές
- **Προβλήματα επίλυσης περιορισμών** όταν τα πεδία τιμών των μεταβλητών παίρνουν συνεχείς τιμές

Ανάλογα με το πόσες μεταβλητές περιλαμβάνει ένας περιορισμός χαρακτηρίζεται ως:

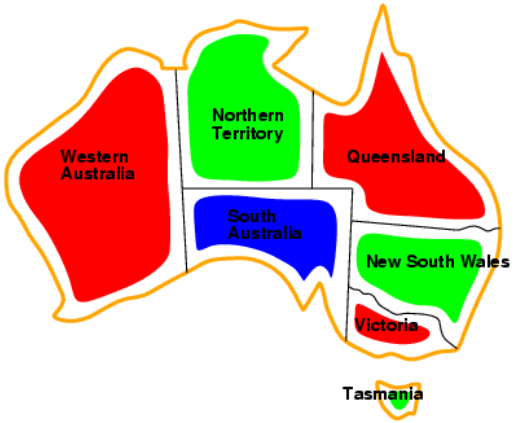
- *μοναδιαίος* (*unary*) όταν περιλαμβάνει μια μεταβλητή,
- *δυαδικός* (*binary*) όταν περιλαμβάνει δύο μεταβλητές ή
- *ανώτερης* τάξης (*higher order*) όταν περιλαμβάνει περισσότερες.

Παραδείγματα "toy" CP προβλημάτων

- 8-Queens
- Χρωματισμός χάρτη
- Κρυπταριθμητικός γρίφος



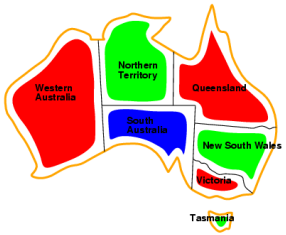
```
  SEND
+ MORE
--------
 MONEY
```



Χρωματισμός χάρτη

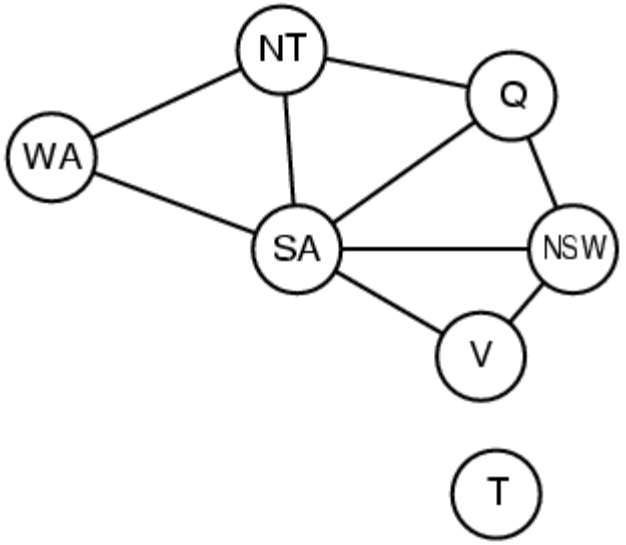


- Μεταβλητές: WA, NT, SA, Q, NSW, V, T
- Πεδίο τιμών: {**Red**, **Green**, **Blue**}
- Περιορισμοί: θα πρέπει γειτονικές περιοχές να χρωματιστούν με διαφορετικό χρώμα.
  - WA ≠ NT, WA ≠ SA
  - NT ≠ SA, NT ≠ Q,
  - SA ≠ Q, SA ≠ NSW, SA ≠ W
  - Q ≠ NSW
  - NSW ≠ V



Γράφημα περιορισμών

- Το γράφημα περιορισμών είναι μια **αφαίρεση** του προβλήματος.
- Οι κόμβοι είναι οι μεταβλητές και οι ακμές είναι οι περιορισμοί.
- Δυαδικό πρόβλημα ικανοποίησης περιορισμών:
  - Κάθε περιορισμός συσχετίζει 2 μεταβλητές.



WA ≠ NT, WA ≠ SA
NT ≠ SA, NT ≠ Q,
SA ≠ Q, SA ≠ NSW, SA ≠ W
Q ≠ NSW
NSW ≠ V

Είδη περιορισμών

- **Μοναδιαίοι περιορισμοί**: αφορούν 1 μόνο μεταβλητή
- **Δυαδικοί περιορισμοί**: αφορούν ζεύγη μεταβλητών
- **Περιορισμοί υψηλότερης τάξης**: αφορούν 3 ή περισσότερες μεταβλητές

- Όλοι οι περιορισμοί υψηλότερης τάξης μπορούν να μετατραπούν σε δυαδικούς περιορισμούς (binarization).

Παράδειγμα Προβλήματος Ικανοποίησης Περιορισμών

- Έστω ότι πρέπει να ορισθεί η σειρά με την οποία θα εισαχθούν τα προϊόντα A, B, C, D μέσα σε ένα βιομηχανικό μύλο. Τα προϊόντα εισάγονται διαδοχικά. Λόγω κάποιων παρασκευαστικών παραμέτρων του τελικού προϊόντος, το προϊόν A πρέπει να εισαχθεί στο μύλο μετά από το D, το C πριν από το B, και το B πριν από το A.

Παράδειγμα Προβλήματος Ικανοποίησης Περιορισμών

- Έστω ότι πρέπει να ορισθεί η σειρά με την οποία θα εισαχθούν τα προϊόντα A, B, C, D μέσα σε ένα βιομηχανικό μύλο. Τα προϊόντα εισάγονται διαδοχικά. Λόγω κάποιων παρασκευαστικών παραμέτρων του τελικού προϊόντος, το προϊόν A πρέπει να εισαχθεί στο μύλο μετά από το D, το C πριν από το B, και το B πριν από το A.

Παράδειγμα: **σειρά εκτέλεσης εργασιών A, B, C, D** σε ένα εργοστάσιο

- V_A, V_B, V_C, $V_D \in \{1, 2, 3, 4\}$: η σειρά κάθε εργασίας

Παράδειγμα Προβλήματος Ικανοποίησης Περιορισμών

- Έστω ότι πρέπει να ορισθεί η σειρά με την οποία θα εισαχθούν τα προϊόντα A, B, C, D μέσα σε ένα βιομηχανικό μύλο. Τα προϊόντα εισάγονται διαδοχικά. Λόγω κάποιων παρασκευαστικών παραμέτρων του τελικού προϊόντος, το προϊόν A πρέπει να εισαχθεί στο μύλο μετά από το D, το C πριν από το B, και το B πριν από το A.

Παράδειγμα: **σειρά εκτέλεσης εργασιών A, B, C, D** σε ένα εργοστάσιο

- $V_A, V_B, V_C, V_D \in \{1, 2, 3, 4\}$: η σειρά κάθε εργασίας
- $V_A \neq V_B, V_A \neq V_C, V_A \neq V_D, V_B \neq V_C, V_B \neq V_D, V_C \neq V_D$ *(έτσι ώστε να μην πάρουν δύο προϊόντα την ίδια σειρά)*

Παράδειγμα Προβλήματος Ικανοποίησης Περιορισμών

- Έστω ότι πρέπει να ορισθεί η σειρά με την οποία θα εισαχθούν τα προϊόντα A, B, C, D μέσα σε ένα βιομηχανικό μύλο. Τα προϊόντα εισάγονται διαδοχικά. Λόγω κάποιων παρασκευαστικών παραμέτρων του τελικού προϊόντος, το προϊόν A πρέπει να εισαχθεί στο μύλο μετά από το D, το C πριν από το B, και το B πριν από το A.

Παράδειγμα: **σειρά εκτέλεσης εργασιών A, B, C, D** σε ένα εργοστάσιο

- $V_A, V_B, V_C, V_D \in \{1, 2, 3, 4\}$: η σειρά κάθε εργασίας
- $V_A \neq V_B, V_A \neq V_C, V_A \neq V_D, V_B \neq V_C, V_B \neq V_D, V_C \neq V_D$ *(έτσι ώστε να μην πάρουν δύο προϊόντα την ίδια σειρά)*
- $V_A > V_D, V_C < V_B, V_B < V_A$ (η A αφού γίνει η D, η C πριν γίνει η B, η B πριν γίνει η A)

Ικανοποίηση περιορισμών -Παράδειγμα

- Παράδειγμα: **σειρά εκτέλεσης εργασιών A, B, C, D** σε ένα εργοστάσιο
  - $V_A, V_B, V_C, V_D \in \{1, 2, 3, 4\}$: η σειρά κάθε εργασίας
  - $V_A \neq V_B, V_A \neq V_C, V_A \neq V_D, V_B \neq V_C, V_B \neq V_D, V_C \neq V_D$ *(έτσι ώστε να μην πάρουν δύο προϊόντα την ίδια σειρά)*
  - $V_A > V_D, V_C < V_B, V_B < V_A$ (η A αφού γίνει η D, η C πριν γίνει η B, η B πριν γίνει η A)
- και το πρόβλημα έχει τις ακόλουθες τρεις δυνατές λύσεις:
- **$V_A = 4, V_B = 2, V_C = 1, V_D = 3$** Δηλαδή η σειρά είναι: **C, B, D, A**
- **$V_A = 4, V_B = 3, V_C = 1, V_D = 2$** Δηλαδή η σειρά είναι: **C, D, B, A**
- **$V_A = 4, V_B = 3, V_C = 2, V_D = 1$** Δηλαδή η σειρά είναι: **D, C, B, A**

Παραγωγή και δοκιμή

Παραγωγή και δοκιμή

- Μια **γεννήτρια** παράγει **υποψήφιες λύσεις** και **ελέγχουμε** αν παραβιάζουν περιορισμούς (ελέγχει αν οι λύσεις ικανοποιούν τους περιορισμούς).
- Η μέθοδος παράγει διαδοχικά τις ακόλουθες λύσεις:
  - $V_A = 1, V_B = 1, V_C = 1, V_D = 1$
  - $V_A = 1, V_B = 1, V_C = 1, V_D = 2$ …
- Ο ελεγκτής ελέγχει τις παραγόμενες λύσεις απορρίπτοντας όσες δεν ικανοποιούν τους περιορισμούς.
- Αν η γεννήτρια χρησιμοποιεί ως πληροφορία ότι το προϊόν Α παρασκευάζεται πάντα τελευταίο, παράγει μόνο τις λύσεις στις οποίες η τιμή της μεταβλητής V_A είναι 4:

$$\mathbf{V_A = 4, V_B = 1, V_C, = 1, V_D = 1}$$

$$\mathbf{V_A = 4, V_B = 2, V_C, = 1, V_D = 1}$$

- Οι πιθανές λύσεις μειώνονται έτσι από $4^4 = 256$ που ήταν στην προηγούμενη περίπτωση σε $4^3 = 64$.

Χρήση κλασικών αλγορίθμων

- Τα προβλήματα ικανοποίησης περιορισμών μπορούν να αναπαρασταθούν **με χώρο αναζήτησης**.
  - **κατάσταση**: περιλαμβάνει τις μεταβλητές, δεν έχουν όλες τιμές.
  - **μεταβάσεις**: ανάθεση τιμής σε κάποια μεταβλητή.
- Άρα **μπορούν να χρησιμοποιηθούν** οι προηγούμενοι αλγόριθμοι αναζήτησης.
- Όμως το **μεγάλο μέγεθος του χώρου αναζήτησης** κάνει συχνά μη πρακτική τη χρήση αυτών των αλγορίθμων.
- Έχουν προταθεί **ειδικοί αλγόριθμοι** για προβλήματα ικανοποίησης περιορισμών.

Παρατηρήσεις για τα προβλήματα CP ως προβλήματα αναζήτησης

- Αν υπάρχουν ***n*** μεταβλητές, το δένδρο εκτείνεται μέχρι το βάθος ***n*** και γι' αυτό το λόγο **στα προβλήματα CP είναι δημοφιλείς οι αλγόριθμοι αναζήτησης πρώτα κατά βάθος**.
- Αν το πεδίο τιμών κάθε μεταβλητής έχει ***d*** τιμές τότε ο αριθμός των δυνατών αναθέσεων είναι $O(d^n)$ δηλαδή είναι εκθετικός ως προς τον αριθμό των μεταβλητών ***n***.

Αναζήτηση με οπισθοδρόμηση

Αναζήτηση με οπισθοδρόμηση

- Ο βασικός αλγόριθμος αναζήτησης υλοποιεί μια αναζήτηση που **συστηματικά** δημιουργεί όλες τις πιθανές αναθέσεις, μια προς μια, σε μια προσπάθεια **μετασχηματισμού μιας μερικής λύσης σε πλήρη λύση**.
- Πρόκειται για έναν **αλγόριθμο αναζήτησης πρώτα κατά βάθος (DFS)** στον οποίο σε κάθε κίνηση γίνεται ανάθεση μιας τιμής σε μια μεταβλητή.
- Αν ο αλγόριθμος αναζήτησης συναντήσει αδιέξοδο, τότε οπισθοδρομεί και εξετάζει τις τελευταίες αναθέσεις που έχει κάνει.

Ψευδοκώδικας αναζήτησης με οπισθοδρόμηση

```
function BACKTRACKING-SEARCH( csp) returns a solution, or failure
    return RECURSIVE-BACKTRACKING({}, csp)

function RECURSIVE-BACKTRACKING( assignment,csp) returns a solution, or
failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE( Variables[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment according to Constraints[csp] then
            add { var = value } to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failue then return result
            remove { var = value } from assignment
    return failure
```

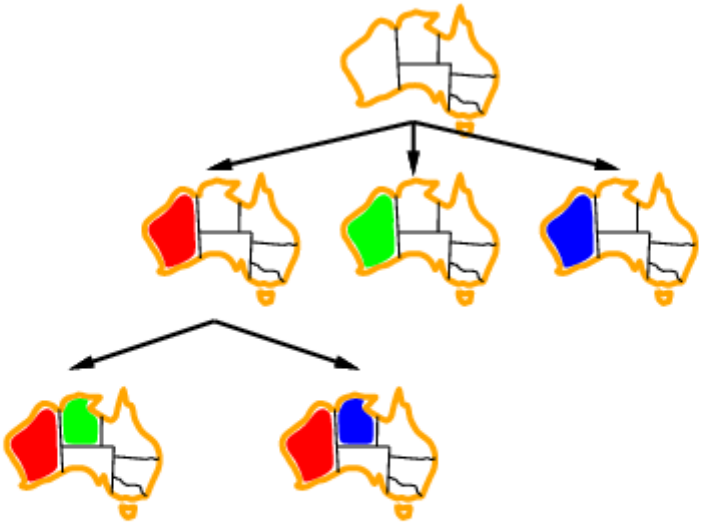
Αναζήτηση με οπισθοδρόμηση



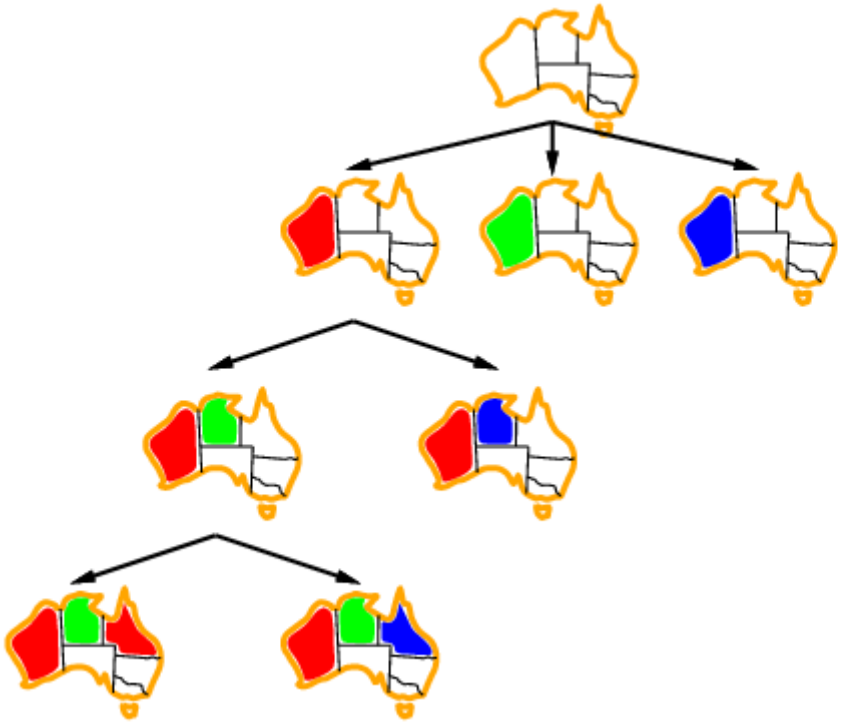
Αναζήτηση με οπισθοδρόμηση



Αναζήτηση με οπισθοδρόμηση



Αναζήτηση με οπισθοδρόμηση



Βασικοί τρόποι βελτίωσης της αναζήτησης με οπισθοδρόμηση

- **Πρώιμος έλεγχος**
- **Διάταξη μεταβλητών**
  - Ποια θα είναι η επόμενη μεταβλητή που θα επιλέγει για ανάθεση τιμής;
  - Με ποια σειρά θα εξεταστούν οι τιμές της μεταβλητής που επιλέχθηκε;
- **Διάδοση περιορισμών** (ανίχνευση μελλοντικής αποτυχίας από νωρίς)
  - Συνέπεια τόξου (αλγόριθμος AC-3)
  - Συνέπεια μονοπατιού (path consistency)

Πρώιμος έλεγχος (forward checking)

Πρώιμος έλεγχος (forward checking)

- Στον πρώιμο έλεγχο **κάθε φορά που ανατίθεται τιμή σε μια μεταβλητή, ελέγχονται οι μεταβλητές που συνδέονται με τη μεταβλητή αυτή και αφαιρούνται τιμές οι οποίες δεν είναι πλέον έγκυρες από τα πεδία τους**.
- Στον πρώιμο έλεγχο μεταφέρεται πληροφορία από τις μεταβλητές στις οποίες έχει ανατεθεί τιμή σε εκείνες στις οποίες δεν έχει ανατεθεί (δεν μεταφέρεται πληροφορία μεταξύ μεταβλητών που δεν έχουν λάβει ακόμα τιμή).

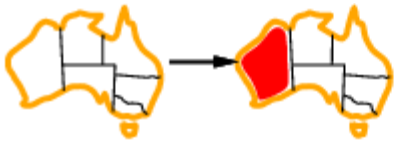


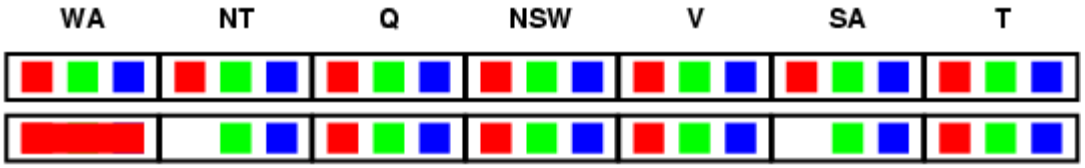




Πρώιμος έλεγχος (forward checking)

- Στον πρώιμο έλεγχο **κάθε φορά που ανατίθεται τιμή σε μια μεταβλητή, ελέγχονται οι μεταβλητές που συνδέονται με τη μεταβλητή αυτή και αφαιρούνται τιμές οι οποίες δεν είναι πλέον έγκυρες από τα πεδία τους**.
- Στον πρώιμο έλεγχο μεταφέρεται πληροφορία από τις μεταβλητές στις οποίες έχει ανατεθεί τιμή σε εκείνες στις οποίες δεν έχει ανατεθεί (δεν μεταφέρεται πληροφορία μεταξύ μεταβλητών που δεν έχουν λάβει ακόμα τιμή).



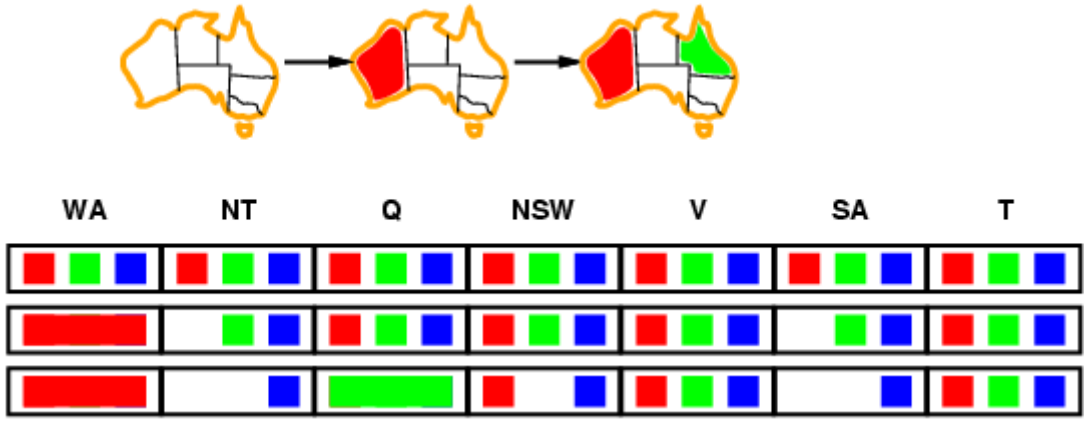




Πρώιμος έλεγχος (forward checking)

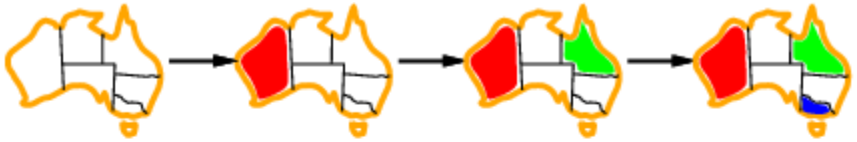
- Στον πρώιμο έλεγχο **κάθε φορά που ανατίθεται τιμή σε μια μεταβλητή, ελέγχονται οι μεταβλητές που συνδέονται με τη μεταβλητή αυτή και αφαιρούνται τιμές οι οποίες δεν είναι πλέον έγκυρες από τα πεδία τους**.
- Στον πρώιμο έλεγχο μεταφέρεται πληροφορία από τις μεταβλητές στις οποίες έχει ανατεθεί τιμή σε εκείνες στις οποίες δεν έχει ανατεθεί (δεν μεταφέρεται πληροφορία μεταξύ μεταβλητών που δεν έχουν λάβει ακόμα τιμή).





Πρώιμος έλεγχος (forward checking)

- Στον πρώιμο έλεγχο **κάθε φορά που ανατίθεται τιμή σε μια μεταβλητή, ελέγχονται οι μεταβλητές που συνδέονται με τη μεταβλητή αυτή και αφαιρούνται τιμές οι οποίες δεν είναι πλέον έγκυρες από τα πεδία τους**.
- Στον πρώιμο έλεγχο μεταφέρεται πληροφορία από τις μεταβλητές στις οποίες έχει ανατεθεί τιμή σε εκείνες στις οποίες δεν έχει ανατεθεί (δεν μεταφέρεται πληροφορία μεταξύ μεταβλητών που δεν έχουν λάβει ακόμα τιμή).







Πρώιμος έλεγχος (forward checking)

- Στον πρώιμο έλεγχο **κάθε φορά που ανατίθεται τιμή σε μια μεταβλητή, ελέγχονται οι μεταβλητές που συνδέονται με τη μεταβλητή αυτή και αφαιρούνται τιμές οι οποίες δεν είναι πλέον έγκυρες από τα πεδία τους**.
- Στον πρώιμο έλεγχο μεταφέρεται πληροφορία από τις μεταβλητές στις οποίες έχει ανατεθεί τιμή σε εκείνες στις οποίες δεν έχει ανατεθεί (δεν μεταφέρεται πληροφορία μεταξύ μεταβλητών που δεν έχουν λάβει ακόμα τιμή).







Αδιέξοδο που δεν το ανιχνεύει ο πρώιμος έλεγχος (δεν μπορεί SA και NT να είναι **B**)

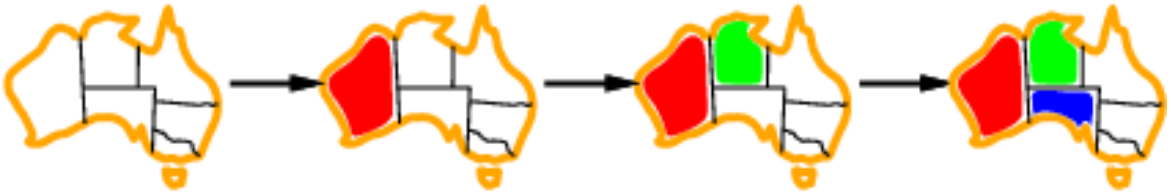
Βασικοί τρόποι βελτίωσης της αναζήτησης με οπισθοδρόμηση

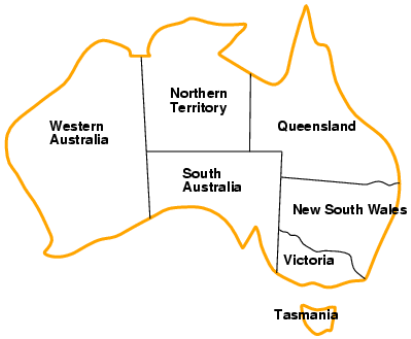
- **Πρώιμος έλεγχος**
- **Διάταξη μεταβλητών**
  - Ποια θα είναι η επόμενη μεταβλητή που θα επιλέγει για ανάθεση τιμής;
  - Με ποια σειρά θα εξεταστούν οι τιμές της μεταβλητής που επιλέχθηκε;
- **Διάδοση περιορισμών** (ανίχνευση μελλοντικής αποτυχίας από νωρίς)
  - Συνέπεια τόξου (αλγόριθμος AC-3)
  - Συνέπεια μονοπατιού (path consistency)

Διάταξη μεταβλητών με τον MRV

- **Ευρετικός μηχανισμός ελάχιστων υπολοίπων τιμών (MRV=Minimum Remaining Values)**: επιλογή της μεταβλητής με το μικρότερο αριθμό από έγκυρες εναπομείνασες τιμές (fail fast ordering).
- Σκεπτικό: Τελικά θα πρέπει να έχουν λάβει τιμές όλες οι μεταβλητές, οπότε ας εξεταστούν πρώτες εκείνες που θα οδηγήσουν νωρίτερα σε μη έγκυρες καταστάσεις και θα προκαλέσουν κλάδεμα του δένδρου.

- Παράδειγμα: Χρωματισμός του χάρτη της Αυστραλίας με 3 χρώματα:
  - Έστω ότι έχουν γίνει οι αναθέσεις: WA=**R**, NT=**G**
  - Πεδία τιμών:
    - SA ϵ {**B**}
    - Q ϵ {**R**,**B**}
    - NSW, V, T ϵ {**R**,**G**,**B**}



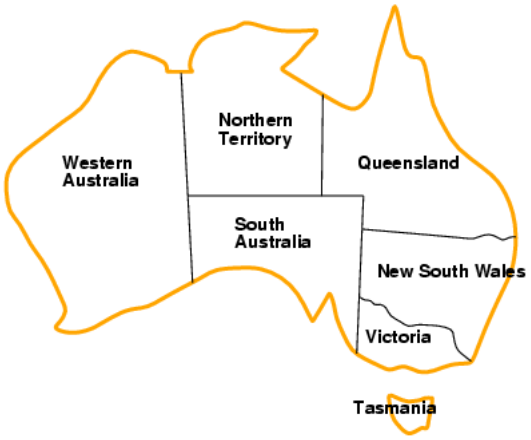


Διάταξη τιμών για τη μεταβλητή στην οποία θα ανατεθεί τιμή

- **Ευρετικός μηχανισμός λιγότερο δεσμευτικής τιμής (LCV=Least Constraining Value)**: έχοντας επιλέξει τη μεταβλητή στην οποία θα ανατεθεί τιμή επιλέγεται η τιμή που αποκλείει το μικρότερο αριθμό από τιμές των άλλων μεταβλητών.
- Σκεπτικό: Ψάχνουμε επιθετικά για λύση, δίνοντας ευελιξία στις αναθέσεις μεταβλητών για τις επόμενες τιμές.

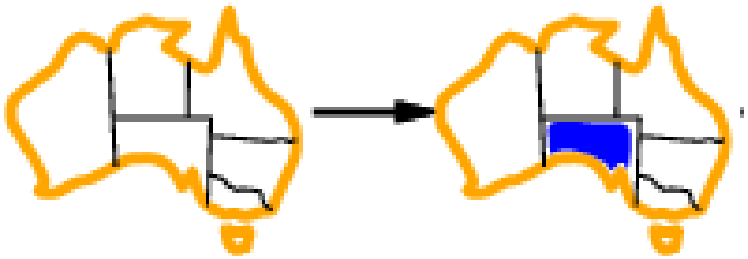
- Παράδειγμα: Χρωματισμός του χάρτη της Αυστραλίας με 3 χρώματα:
  - Έστω ότι έχουν γίνει οι αναθέσεις: WA=**R**, NT=**G** και ότι η επόμενη περιοχή είναι η Q με πεδίο τιμών {**R**,**B**}
  - Ο ευρετικός μηχανισμός LCV θα επιλέξει Q=**R** και όχι Q=**B** διότι η δεύτερη επιλογή θα έχει ως αποτέλεσμα η περιοχή SA να μην έχει τιμές.





Ευρετικός μηχανισμός βαθμού

- Ο **ευρετικός μηχανισμός βαθμού (degree heuristic)** μπορεί να χρησιμοποιηθεί όταν ο MRV παράγει ισοδύναμες λύσεις (δηλαδή με το ίδιο πλήθος πιθανών τιμών).
- Επιλέγει τη μεταβλητή που συμμετέχει στους περισσότερους περιορισμούς σε σχέση με τις άλλες μεταβλητές στις οποίες δεν έχει ανατεθεί τιμή.
- Π.χ. στο πρόβλημα του χρωματισμού του χάρτη της Αυστραλίας ο ευρετικός μηχανισμός βαθμού θα επιλέξει αρχικά την περιοχή SA.



Παραγωγή και Δοκιμή

Παραγωγή και δοκιμή

- Απλές γνώσεις του προβλήματος μπορούν να ενσωματωθούν στη γεννήτρια (**ενήμερη γεννήτρια**)
  - π.χ. πάντα διαφορετικές τιμές μεταβλητών.
- **Απλή** μέθοδος αλλά συνήθως **μη αποδοτική** σε προβλήματα ικανοποίησης περιορισμών (μεγάλος αριθμός υποψηφίων λύσεων).
- Ο κλασικός αλγόριθμος **Παραγωγής και Δοκιμής** έχει **τεράστιο χώρο αναζήτησης** οπότε έχουν προταθεί οι **αλγόριθμοι επιδιόρθωσης:**
  - **Αναρρίχηση λόφου**
  - **Ευριστικός Αλγόριθμος ελαχίστων συγκρούσεων**

Παραγωγή και δοκιμή

- Ο κλασικός αλγόριθμος **Παραγωγής και Δοκιμής** έχει **τεράστιο χώρο αναζήτησης** οπότε έχουν προταθεί οι **αλγόριθμοι επιδιόρθωσης:**
  - **Αναρρίχηση λόφου**
  - **Ευριστικός Αλγόριθμος ελαχίστων συγκρούσεων**

Αναρρίχηση Λόφου για επιδιόρθωση

1. Ανέθεσε στις μεταβλητές τυχαίες τιμές από τα πεδία τιμών τους.
2. Αν οι τιμές των μεταβλητών δεν παραβιάζουν τους περιορισμούς του προβλήματος τότε επέστρεψε τις τιμές αυτές ως λύση.
3. Εξέτασε για κάθε μεταβλητή όλες τις δυνατές τιμές που μπορεί να πάρει.
   i. Αν κάποια από τις τιμές που εξετάστηκαν ελαχιστοποιεί το πλήθος των περιορισμών που παραβιάζονται, ανέθεσε την τιμή της στην αντίστοιχη μεταβλητή και επέστρεψε στο βήμα 2.
   ii. Αν δε υπάρχει τιμή που να ελαχιστοποιεί το πλήθος των περιορισμών, τότε επέστρεψε στο βήμα 1 (τοπικό ελάχιστο– αλγόριθμος ξεκινά από μια νέα τυχαία ανάθεση τιμών).

Μειονεκτήματα:

- Εξετάζει ένα μεγάλο πλήθος "γειτονικών" καταστάσεων πριν επιλέξει την επόμενη τιμή η οποία θα μεταβληθεί, και
- Μπορεί να "πέσει" σε τοπικό ελάχιστο, (όπου καμιά μεταβολή στις τιμές δε δίνει καλύτερη λύση.)

Παραγωγή και δοκιμή

- Ο κλασικός αλγόριθμος **Παραγωγής και Δοκιμής** έχει **τεράστιο χώρο αναζήτησης** οπότε έχουν προταθεί οι **αλγόριθμοι επιδιόρθωσης:**
  - **Αναρρίχηση λόφου**
  - **Ευριστικός Αλγόριθμος ελαχίστων συγκρούσεων**

Ευριστικός αλγόριθμος ελαχίστων συγκρούσεων (min conflicts heuristic)

1. Ανάθεσε στις μεταβλητές **τυχαίες τιμές** από τα πεδία τιμών τους.
2. Αν οι τιμές **δεν παραβιάζουν** τους περιορισμούς, επέστρεψε τις τιμές και σταμάτα.
3. Εξέτασε για μια **τυχαία μεταβλητή** όλες τις δυνατές τιμές της.
   - Αν κάποια τιμή **μειώνει** το πλήθος των περιορισμών που παραβιάζονται, κράτα αυτήν την τιμή.
   - Αν δεν υπάρχει τέτοια τιμή, διάλεξε μια νέα τιμή που να διατηρεί τον **ίδιο αριθμό περιορισμών**.
   - Αν δεν υπάρχει ούτε τέτοια τιμή, άφησε την **παλιά**.
4. Επέστρεψε στο βήμα 2.

Ευρεστικός Αλγόριθμος ελαχίστων συγκρούσεων

- Ο αλγόριθμος ξεκινά από μια τυχαία ανάθεση τιμών και μεταβάλλει την τιμής μιας μεταβλητής έτσι ώστε η νέα τιμή να παραβιάζει λιγότερους περιορισμούς. Η μεταβλητή επιλέγεται τυχαία, από το σύνολο των μεταβλητών που είναι σε σύγκρουση με τους περιορισμούς
- Εξετάζει **μικρό αριθμό γειτονικών κόμβων** σε κάθε επανάληψη.
- Πρόβλημα **εγκλωβισμού σε τοπικό ελάχιστο**.
- Αποτελεί παράδειγμα *αλγορίθμου επιδιόρθωσης*.

Κλασσικοί Αλγόριθμοι Αναζήτησης

Κλασσικοί Αλγόριθμοι Αναζήτησης

- Οι αλγόριθμοι αναζήτησης που παρουσιάστηκαν σε προηγούμενες ενότητες είναι δυνατό να χρησιμοποιηθούν και για την επίλυση των προβλημάτων ικανοποίησης περιορισμών.
- **Αλγόριθμοι Ελέγχου Συνέπειας**
- Για κάθε περιορισμό αφαίρεσε από τα πεδία τιμών των μεταβλητών τις τιμές εκείνες που δεν μπορούν να συμμετέχουν στην τελική λύση.
- Στο μειωμένο χώρο αναζήτησης που προκύπτει από το προηγούμενο βήμα εφάρμοσε έναν κλασσικό αλγόριθμο αναζήτησης για να βρεθεί η λύση.
  - Σε κάθε βήμα (ανάθεση τιμής) αυτής της αναζήτησης εφάρμοσε ξανά τον αλγόριθμο ελέγχου συνέπειας έτσι ώστε να αφαιρεθούν τυχόν τιμές από τα πεδία των μεταβλητών οι οποίες δεν μπορούν να συμμετέχουν στην λύση.

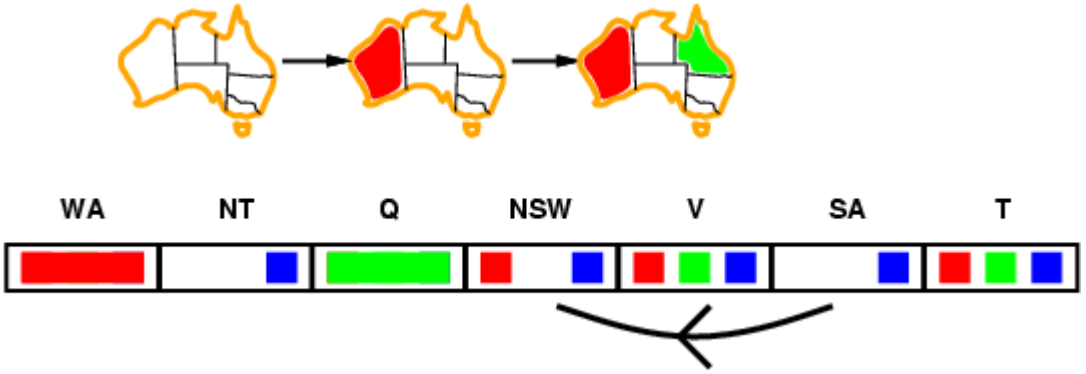
Διάδοση περιορισμών (constraint propagation)

- Ο στόχος της διάδοσης περιορισμών είναι να ανιχνευθούν ακόμα νωρίτερα σε σχέση με τον πρώιμο έλεγχο μελλοντικές αποτυχίες.
- Μια μορφή διάδοσης περιορισμών είναι η **συνέπεια τόξου** (arc consistency).
- Είναι σε θέση να ανιχνεύσει καταστάσεις στις οποίες οι τιμές που έχουν απομείνει σε μεταβλητές στις οποίες δεν έχουν ανατεθεί ακόμα τιμές δεν είναι έγκυρες για να προκύψει συνολική λύση του προβλήματος.

Συνέπεια τόξου

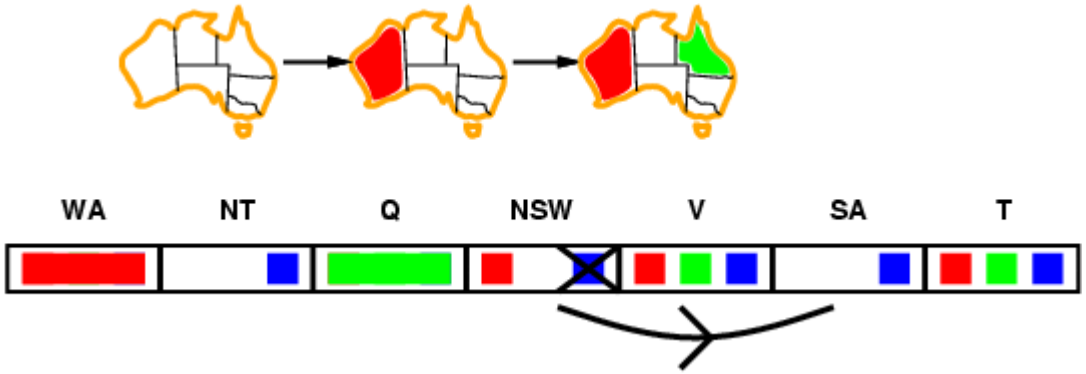
- Σε ένα γράφημα δυαδικών περιορισμών οι μεταβλητές του προβλήματος αποτελούν τους κόμβους ενώ οι περιορισμοί ανάμεσα στις μεταβλητές ορίζουν τόξα ανάμεσα στους κόμβους.
- Ένα τόξο X→Y είναι **συνεπές** αν και μόνο αν για κάθε τιμή x στο **πίσω άκρο του τόξου (ουρά)** υπάρχει τιμή y στο **εμπρός άκρο του τόξου (κεφαλή)** η οποία μπορεί να επιλεγεί χωρίς να παραβιάζεται ο περιορισμός.
  - Αν το X χάσει τουλάχιστον 1 τιμή τότε μπορεί να ελεγχθούν εκ νέου όλοι οι περιορισμοί οι οποίοι δείχνουν προς το X.
- Ο αλγόριθμος συνέπειας τόξου ανιχνεύει μη έγκυρες καταστάσεις **νωρίτερα** από τον πρώιμο έλεγχο.
- Ο αλγόριθμος συνέπειας τόξου μπορεί να εκτελείται μετά από κάθε ανάθεση.

Παράδειγμα συνέπειας τόξων



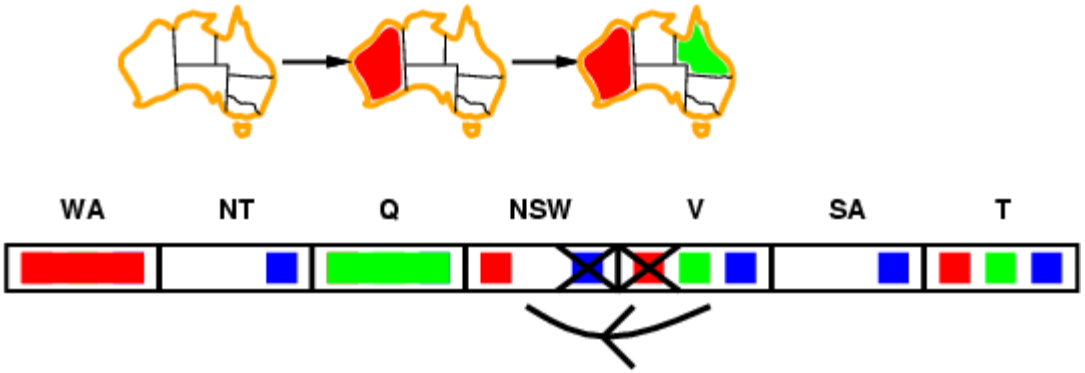


Παράδειγμα συνέπειας τόξων



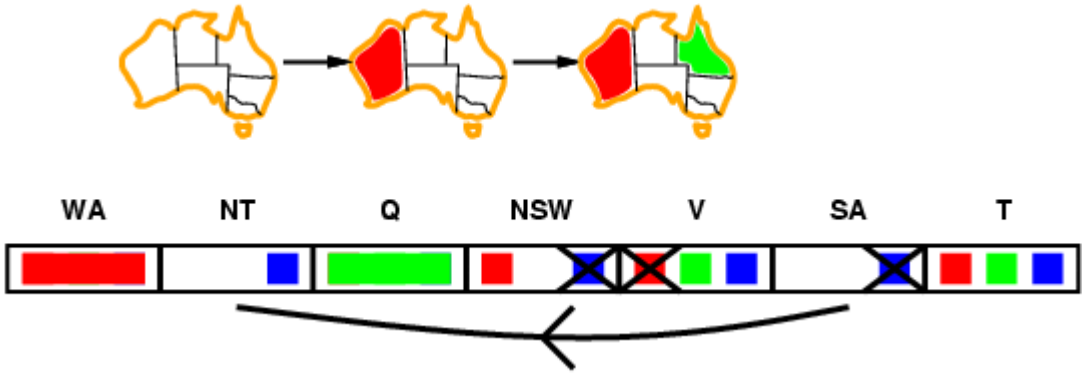


Παράδειγμα συνέπειας τόξων



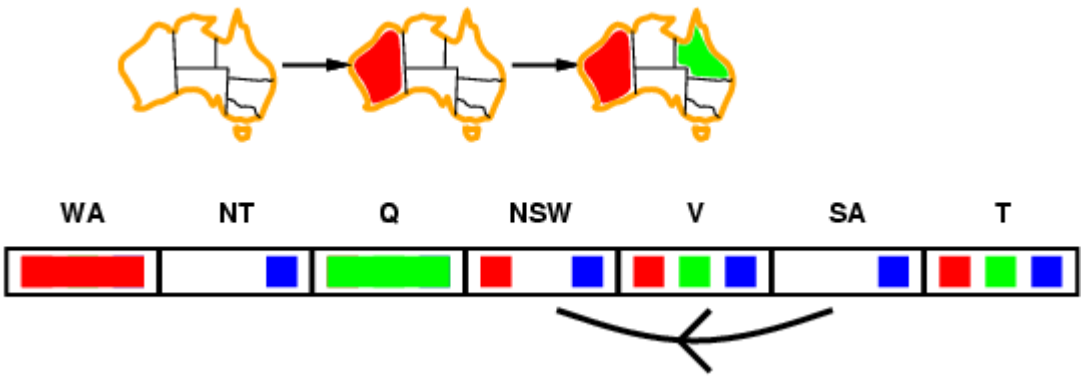


Παράδειγμα συνέπειας τόξων



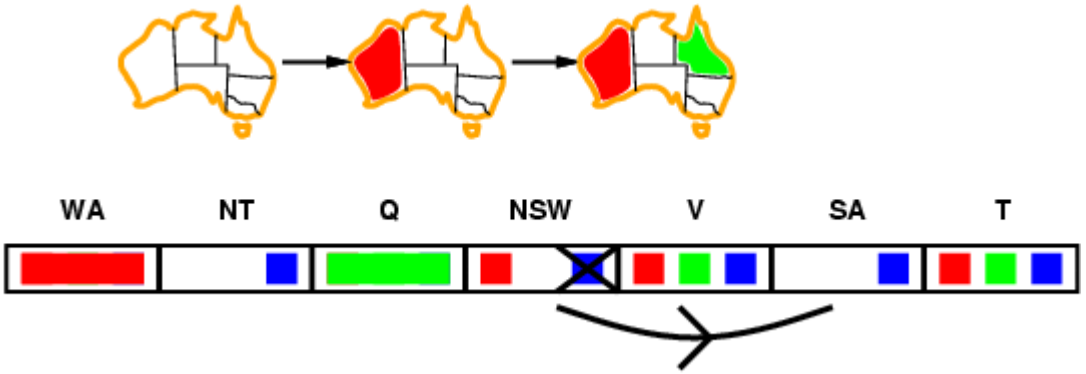


Παράδειγμα συνέπειας τόξων



Το τόξο SA→NSW είναι συνεπές





Το τόξο NSW→SA είναι μη συνεπές, μπορεί να γίνει συνεπές διαγράφοντας το μπλε χρώμα από το πεδίο τιμών του NSW

Παράδειγμα Προβλήματος Ικανοποίησης Περιορισμών

- Έστω ότι πρέπει να ορισθεί η σειρά με την οποία θα εισαχθούν τα προϊόντα A, B, C, D μέσα σε ένα βιομηχανικό μύλο. Λόγω κάποιων παρασκευαστικών παραμέτρων του τελικού προϊόντος, το προϊόν A πρέπει να εισαχθεί στο μύλο μετά από το D, το C πριν από το B, και το B πριν από το A.

Αλγόριθμοι Ελέγχου Συνέπειας
Παράδειγμα (1/3)

$V_A \neq V_B$ (C1) $V_B \neq V_\Gamma$ (C4) $V_A > V_\Delta$ (C7)

$V_A \neq V_\Gamma$ (C2) $V_B \neq V_\Delta$ (C5) $V_\Gamma < V_B$ (C8)

$V_A \neq V_\Delta$ (C3) $V_\Gamma \neq V_\Delta$ (C6) $V_B < V_A$ (C9)

- Τα πεδία τιμών των μεταβλητών:

  $V_A \in \{1,2,3,4\}$

  $V_B \in \{1,2,3,4\}$

  $V_\Gamma \in \{1,2,3,4\}$

  $V_\Delta \in \{1,2,3,4\}$

- Λόγω C9 ($V_B < V_A$), η μεταβλητή V_B δε μπορεί σε καμιά περίπτωση να πάρει την τιμή 4, αλλά ούτε και η V_A να πάρει την τιμή 1:

  $V_A \in \{2,3,4\}$

  $V_B \in \{1,2,3\}$

  $V_\Gamma \in \{1,2,3,4\}$

  $V_\Delta \in \{1,2,3,4\}$

Αλγόριθμοι Ελέγχου Συνέπειας
Παράδειγμα (2/3)

- Λόγω $V_Γ < V_B$ (C8), η $V_Γ$ δεν μπορεί να πάρει την τιμή 3 ούτε και την τιμή 4, ενώ η V_B δε μπορεί να πάρει την τιμή 1:

  $V_A \in \{2,3,4\}$

  $V_B \in \{2,3\}$

  $V_Γ \in \{1,2\}$

  $V_Δ \in \{1,2,3,4\}$

- Λόγω $V_A > V_Δ$ (C7) η $V_Δ$ δεν μπορεί να πάρει την τιμή 4:

  $V_A \in \{2,3,4\}$

  $V_B \in \{2,3\}$

  $V_Γ \in \{1,2\}$

  $V_Δ \in \{1,2,3\}$

Αλγόριθμοι Ελέγχου Συνέπειας
Παράδειγμα (3/3)

- Το πεδίο της V_B έχει μεταβληθεί, οπότε ο περιορισμός C9 πρέπει να επανεξεταστεί. Λόγω του $V_B < V_A$ (C9) δεν μπορεί να υπάρχει η τιμή 2 στο πεδίο της V_A:

  $V_A \in \{3,4\}$

  $V_B \in \{2,3\}$

  $V_Γ \in \{1,2\}$

  $V_Δ \in \{1,2,3\}$

- Τώρα οι πιθανοί συνδυασμοί γίνονται $2 \times 2 \times 2 \times 3 = 24$, σε σχέση με τους 256 που υπήρχαν αρχικά.
- Ο γράφος που προκύπτει ονομάζεται *γράφος περιορισμών* (*constraint graph*).

Γράφος Περιορισμών

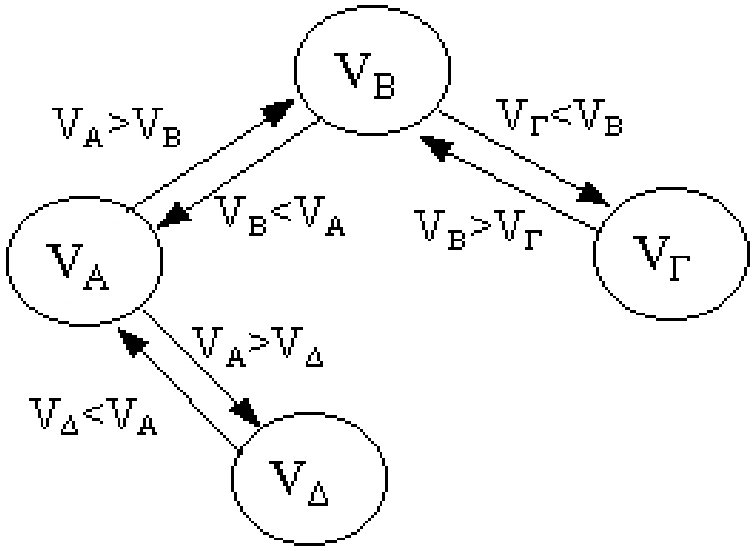
- Η προηγούμενη διαδικασία είναι η βάση των αλγορίθμων ελέγχου συνέπειας (consistency check algorithms).
- Οι αλγόριθμοι που εντάσσονται στην παραπάνω κατηγορία ονομάζονται συνήθως αλγόριθμοι συνέπειας τόξου (Arc Consistency- AC).

- Περιορισμοί:

  $V_A > V_\Delta$

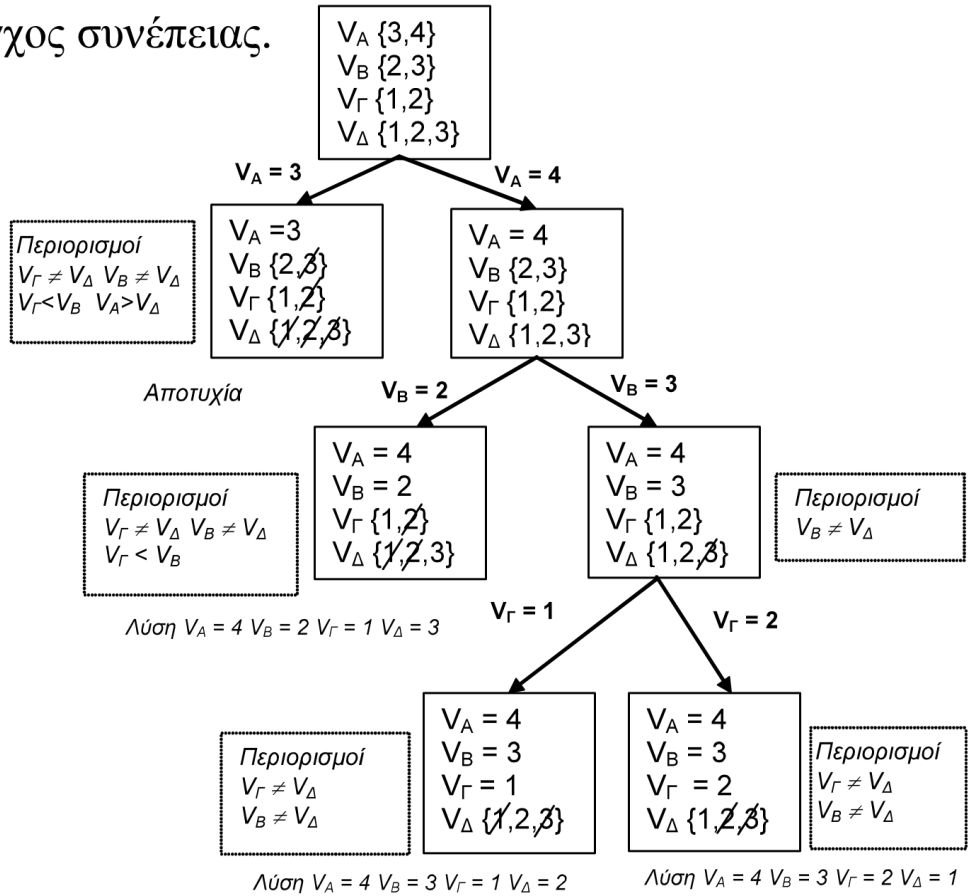
  $V_\Gamma < V_B$

  $V_B < V_A$



Πρόβλημα Βιομηχανικού Μύλου

- Μετά την εφαρμογή των περιορισμών απομένουν οι ακόλουθες τιμές στα πεδία των μεταβλητών: $V_A \in \{3,4\}, V_B \in \{2,3\}, V_\Gamma \in \{1,2\}, V_\Delta \in \{1,2,3\}$
- Σε κάθε βήμα εφαρμόζεται έλεγχος συνέπειας.
- Δένδρο αναζήτησης.



Ο αλγόριθμος AC3 (1/2)

- Έστω οι μεταβλητές V_1, V_2,..V_n με τιμές d_1, d_2,…,d_n από τα πεδία τιμών των μεταβλητών D_1, D_2,…,D_n ($d_1 \in D_1$, $d_2 \in D_2$,… $d_n \in D_n$) και ένα σύνολο περιορισμών $C(V_i,V_j)$ για τις μεταβλητές αυτές, οι οποίοι αναπαριστώνται ως τόξα (V_i,V_j).
- Για συντομία, κάθε τόξο (V_i,V_j) αναφέρεται ως (i,j).
  1. Επανέλαβε τα ακόλουθα βήματα μέχρι το Q να γίνει κενό:
  2. Επέλεξε ένα τόξο (i,j) και διέγραψε το από το Q
  3. Για κάθε τιμή d_i του πεδίου της μεταβλητής V_i έλεγξε αν υπάρχει τουλάχιστον μία τιμή d_j του πεδίου της μεταβλητής V_j τέτοια ώστε να ικανοποιεί το περιορισμό $C(V_i,V_j)$ που αντιστοιχεί στο τοξο (i, j).
  4. Αν δεν υπάρχει τέτοια τιμή d_j τότε αφαίρεσε την τιμή d_i από το πεδίο τιμών της V_i. Αν το πεδίο τιμών της V_i είναι κενό τότε τερμάτισε με αποτυχία.
  5. Αν έχει μεταβληθεί το πεδίο τιμών της V_i τότε πρόσθεσε στο σύνολο Q όλα τα τόξα (k,i), που αντιστοιχούν στους περιορισμούς $C(V_k,V_i)$, για $k \neq i$.

Ο αλγόριθμος AC3 (1/2)

Χαρακτηριστικά του αλγορίθμου AC-3:

- Προϋποθέτει ότι οι περιορισμοί αφορούν μόνο δύο μεταβλητές, είναι δηλαδή *δυαδικοί περιορισμοί* (*binary constraints*).
- Οι μοναδιαίοι περιορισμοί που εμφανίζονται στα προβλήματα περιορισμών αντιμετωπίζονται από τον αλγόριθμο ελέγχου συνέπειας κόμβου (node consistency).
- Για να επιλυθεί ένα πρόβλημα περιορισμών θα πρέπει να χρησιμοποιηθούν αλγόριθμοι ελέγχου συνέπειας τόξου σε συνδυασμό με κάποιον αλγόριθμο αναζήτησης για να βρεθεί η τελική λύση.

Βελτιώσεις:

- Η *αρχή της* συντομότερης αποτυχίας (first fail principle).

Ψευδοκώδικας αλγορίθμου AC-3

function AC-3(*csp*) **returns** the CSP, possibly with reduced domains
  inputs: *csp*, a binary CSP with variables $\{X_1, X_2, \ldots, X_n\}$
  local variables: *queue*, a queue of arcs, initially all the arcs in *csp*

  while *queue* is not empty **do**
    $(X_i, X_j) \leftarrow$ REMOVE-FIRST(*queue*)
    if RM-INCONSISTENT-VALUES(X_i, X_j) **then**
      for each X_k **in** NEIGHBORS[X_i] **do**
        add (X_k, X_i) to *queue*

---

function RM-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff remove a value
  removed $\leftarrow$ *false*
  for each x **in** DOMAIN[X_i] **do**
    if no value y in DOMAIN[X_j] allows (x,y) to satisfy constraint(X_i, X_j)
      then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ *true*
  return *removed*

Μπορεί να αποδειχθεί ότι ο αλγόριθμος AC-3 έχει πολυπλοκότητα ***O(n²d³)*** όπου ***n*** είναι το πλήθος των μεταβλητών και ***d*** το πλήθος των τιμών στο πεδίο τιμών κάθε μεταβλητής.

Παρατηρήσεις σχετικά με τη συνέπεια τόξου

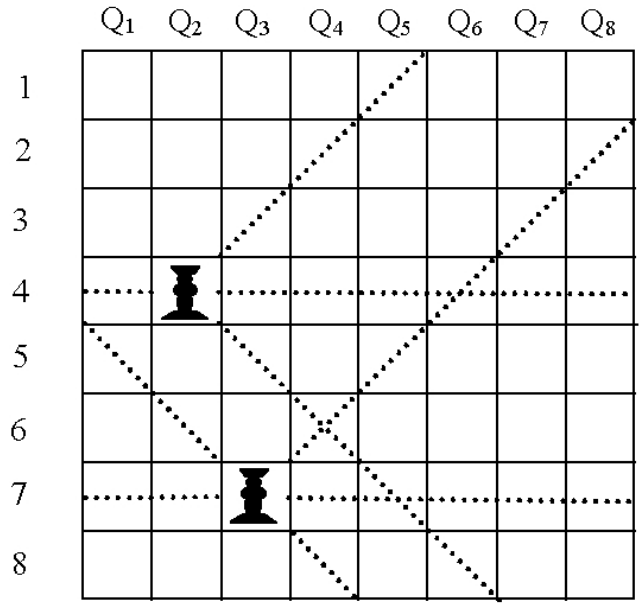
- Απαιτεί επεξεργαστική ισχύ.
- Όταν εφαρμοστεί η συνέπεια τόξου. Εάν:
  - σε κάθε μεταβλητή υπάρχει 1 μόνο διαθέσιμη τιμή ➔ **υπάρχει 1 μόνο λύση**.
  - μια τουλάχιστον μεταβλητή δεν έχει διαθέσιμες τιμές στο πεδίο τιμών της ➔ **δεν υπάρχει λύση**.
  - κάποιες μεταβλητές έχουν πολλές διαθέσιμες τιμές ➔ **υπάρχουν λύσεις που δεν έχουν εντοπιστεί ή δεν υπάρχει λύση**.

K- συνέπεια

- Ένας γράφος περιορισμών είναι K-συνεπής (K-consistent) εάν για κάθε K-1 μεταβλητές που ικανοποιούν τους περιορισμούς υπάρχει μια μεταβλητή K με τέτοιο πεδίο ώστε να ικανοποιούνται ταυτόχρονα όλοι τους οι περιορισμοί που συνδέουν τις K μεταβλητές.
- Ένας γράφος είναι ισχυρά K-συνεπής (strongly K-consistent) εάν για κάθε $L \leq K$, είναι L-συνεπής.
- Σε ένα γράφο με N κόμβους, εάν εξασφαλισθεί ότι ο γράφος είναι ισχυρά N-συνεπής διαγράφονται όλες οι μη αποδεκτές τιμές από τα πεδία των μεταβλητών, όποτε η λύση μπορεί να βρεθεί χωρίς αναζήτηση.

Κ- συνέπεια Παράδειγμα

Πρόβλημα τοποθέτησης 8 βασίλισσες σε μια σκακιέρα 8x8



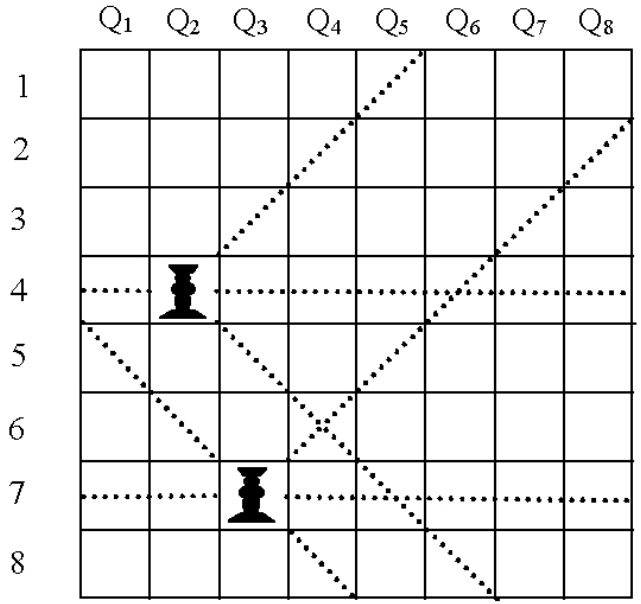
Κ- συνέπεια Παράδειγμα

- Συνθήκη μη απειλής μεταξύ των βασιλισσών:
- Όλες οι βασίλισσες πρέπει να είναι σε διαφορετική γραμμή:

  $\forall$ i, j: $Q_j \neq Q_i$.
- Ισχύουν οι περιορισμοί:

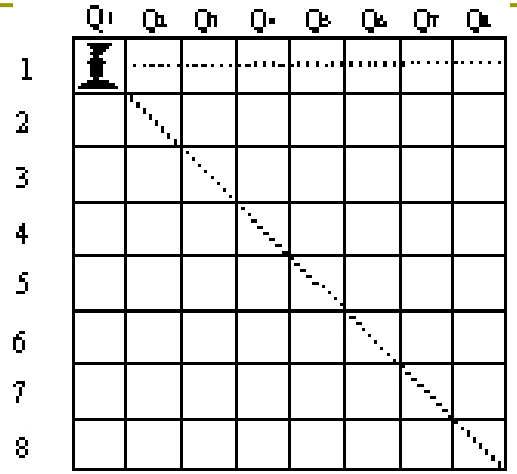
  $Q_j \neq Q_{j+n} + n$ για $n>1$ και $n+j \leq 8$

  $Q_j \neq Q_{j+n} - n$ για $n>1$ και $n+j \leq 8$

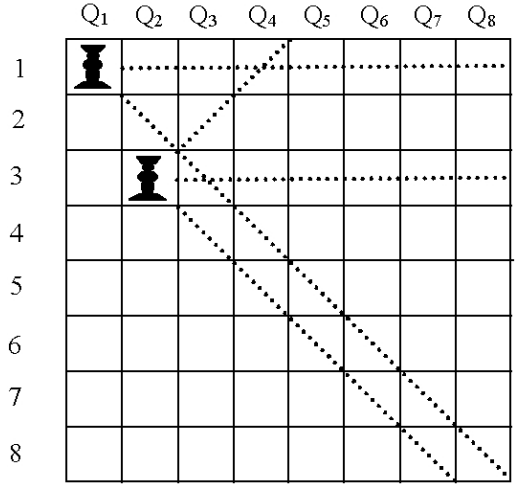


Γραφική αναπαράσταση των περιορισμών (1/2)

- Ανάθεση τιμής στην πρώτη βασίλισσα

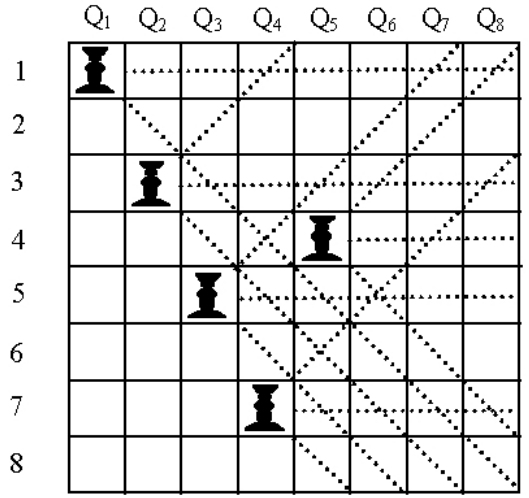


- Ανάθεση τιμών στις δύο πρώτες βασίλισσες



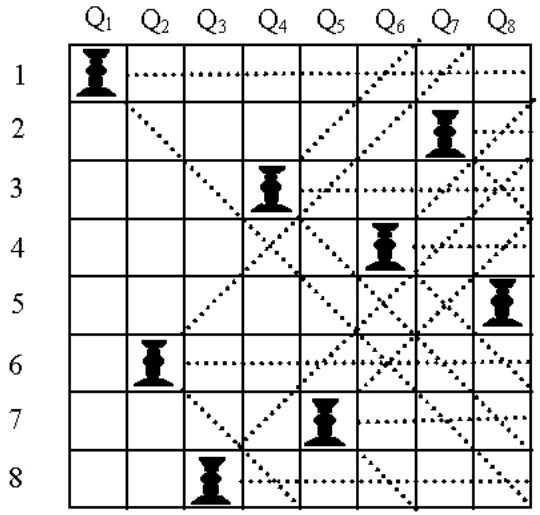
Γραφική αναπαράσταση των περιορισμών (2/2)

- Ανάθεση τιμής που δεν οδηγεί σε λύση



Λύση στο πρόβλημα

- Λύση στο πρόβλημα των 8 βασιλισσών



K-συνέπεια

- **1-συνέπεια**: Το πεδίο τιμών κάθε κόμβου έχει μια τουλάχιστον τιμή που είναι σύμφωνη με τους μοναδιαίους περιορισμούς του προβλήματος.
- **2-συνέπεια (arc consistency)**: Για κάθε ζεύγος κόμβων οποιαδήποτε συνεπής ανάθεση στο ένα κόμβο μπορεί να επεκταθεί και προς τον άλλο.
- **K-συνέπεια**: Για κάθε κ κόμβους οποιαδήποτε συνεπής ανάθεση στους κ-1 κόμβους μπορεί να επεκταθεί και στο κ-οστό κόμβο.
- Υψηλότερο κ στην κ-συνέπεια σημαίνει μεγαλύτερος υπολογιστικός φόρτος.
- K=3 → **συνέπεια διαδρομής** (path consistency).

Τοπική αναζήτηση στα προβλήματα ικανοποίησης περιορισμών

- Οι αλγόριθμοι επαναληπτικής βελτίωσης είναι αλγόριθμοι **τοπικής αναζήτησης**.
- Αντί να δημιουργείται μια λύση σταδιακά, αναθέτοντας τιμές στις μεταβλητές του προβλήματος μια προς μια, οι αλγόριθμοι επαναληπτικής βελτίωσης **ξεκινούν με μια πλήρη λύση** στην οποία είναι πιθανό να παραβιάζονται κάποιοι περιορισμοί.
- Εφαρμόζονται τελεστές (κινήσεις) έτσι ώστε να ανατεθούν νέες τιμές στις μεταβλητές.
- Παράδειγμα: αλγόριθμος ελαχίστων συγκρούσεων (**min conflicts algorithm**)

Αλγόριθμος ελαχίστων συγκρούσεων (Min-Conflicts)

Αλγόριθμος ελαχίστων συγκρούσεων (Min-Conflicts)

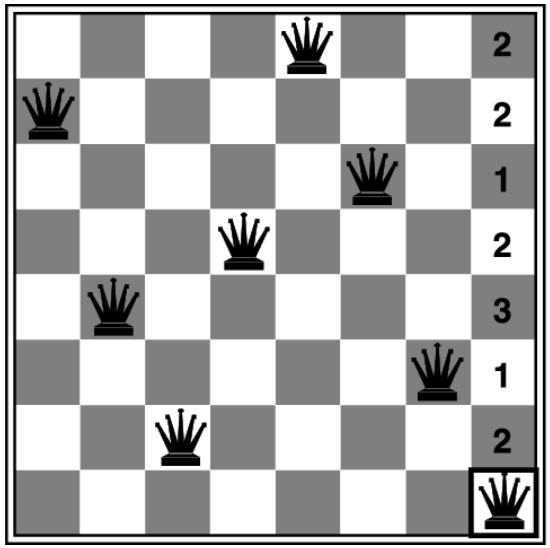
1. Εκκίνηση από μια κατάσταση στην οποία σε κάθε μεταβλητή έχει ανατεθεί μια τιμή.
2. Αν ικανοποιούνται όλοι οι περιορισμοί τότε επέστρεψε τη λύση.
3. Επέλεξε με τυχαίο τρόπο μια μεταβλητή.
4. Επέλεξε τιμή για τη μεταβλητή που θα παραβιάζει τους λιγότερους περιορισμούς
5. Πήγαινε στο βήμα 2

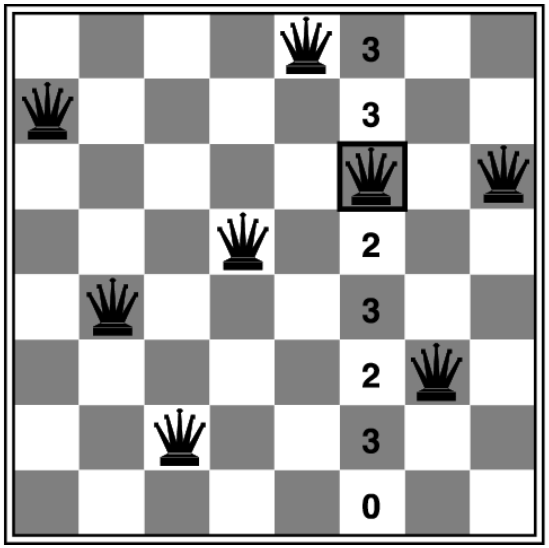
Ψευδοκώδικας αλγορίθμου Min-Conflicts

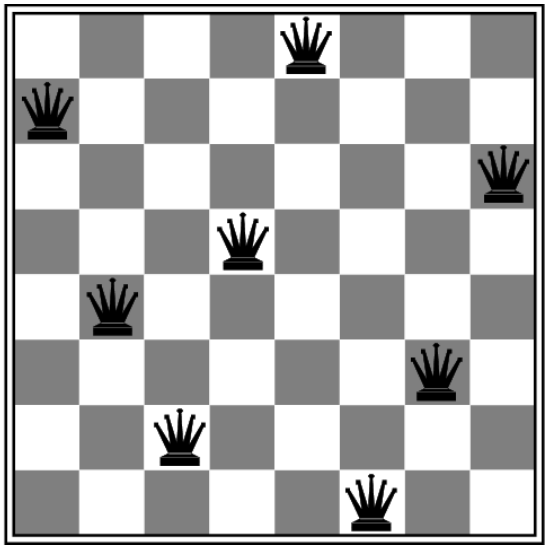
function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or failure
  inputs: *csp*, a constraint satisfaction problem
    max_steps, the number of steps allowed before giving up

  current ← an initial complete assignment for *csp*
  for *i* = 1 to *max_steps* **do**
    if *current* is a solution for *csp* **then return** *current*
    var ← a randomly chosen, conflicted variable from VARIABLES[*csp*]
    value ← the value *v* for *var* that minimizes CONFLICTS(*var*, *v*, *current*, *csp*)
    set *var* = *value* in *current*
  return *failure*

Παράδειγμα εκτέλεσης του Min-Conflicts στο πρόβλημα 8-Queens

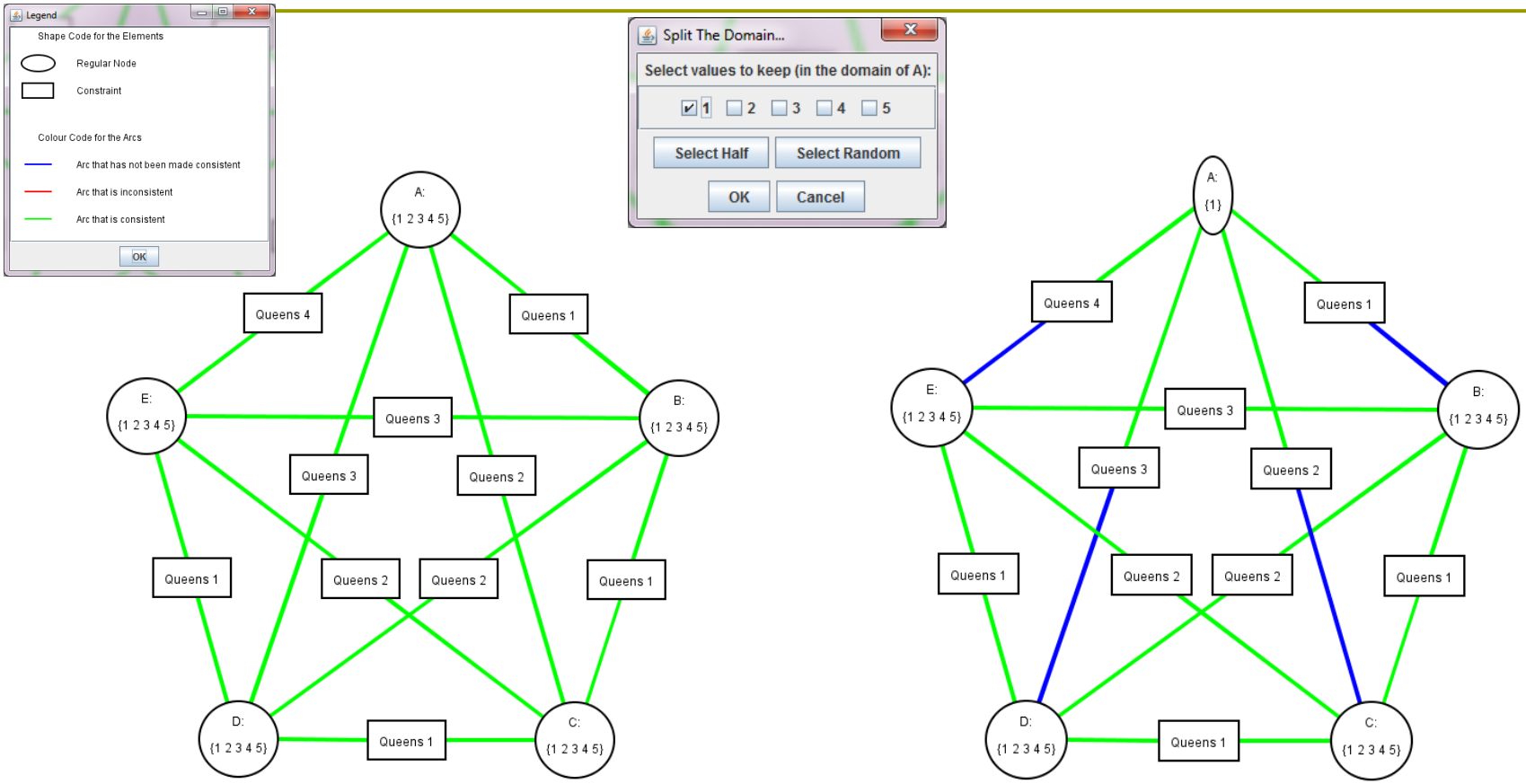






1. Τυχαία επιλογή της βασίλισσας στην 8η στήλη και μετακίνησή της στην 3η σειρά.
2. Τυχαία επιλογή της βασίλισσας στην 6η στήλη και μετακίνησή της στην 8η σειρά.

5-Queens



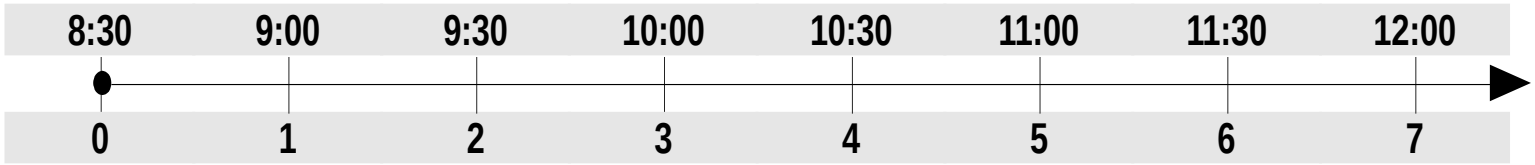
http://aispace.org/constraint/

Παράδειγμα Μετά-Περιορισμών

Παράδειγμα Μετά-Περιορισμών

Εργασίες στα Αυτοκίνητα

	Αυτοκίνητο Α	Αυτοκίνητο Β	Αυτοκίνητο Γ
Ηλεκτρολογικές Εργασίες (Ηλεκτρολόγος)	120min	30min	30min
Μηχανολογικές Εργασίες (Μηχανικός)	60min	90min	60min



Αντιστοίχιση των ημιώρων του πεδίου σε ακέραιες τιμές.

Παράδειγμα Μετά-Περιορισμών

Το πρώτο σύνολο μοναδιαίων περιορισμών:

$A_1 \geq 0 \wedge A_2 \geq 0$ (άφιξη στις 8.30 - χρονική στιγμή 0)

$B_1 \geq 1 \wedge B_2 \geq 1$ (άφιξη στις 9.00 - χρονική στιγμή 1)

$\Gamma_1 \geq 2 \wedge \Gamma_2 \geq 2$ (άφιξη στις 9.30 - χρονική στιγμή 2)



Αρχικά πεδία τιμών των μεταβλητών του προβλήματος, μετά την εφαρμογή των μοναδιαίων περιορισμών

Παράδειγμα Μετά-Περιορισμών

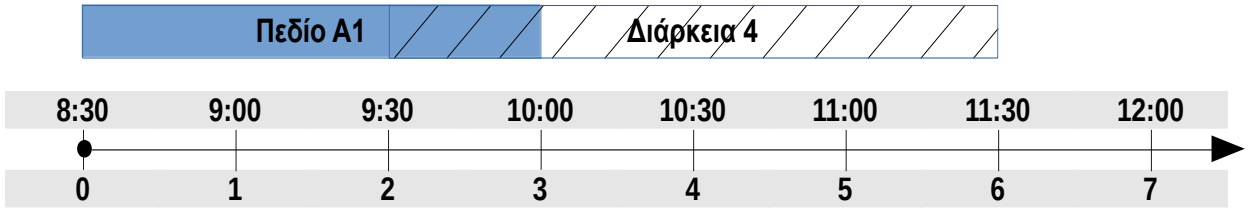
$A_1 + 4 \leq 7$ δηλαδή $A_1 \leq 3$,

$A_2 + 2 \leq 7$

$B_1 + 1 \leq 7 \wedge B_2 + 3 \leq 7$

$\Gamma_1 + 1 \leq 7 \wedge \Gamma_2 + 2 \leq 7$

	Αυτοκίνητο Α	Αυτοκίνητο Β	Αυτοκίνητο Γ
Ηλεκτρολογικές Εργασίες (Ηλεκτρολόγος)	120min	30min	30min
Μηχανολογικές Εργασίες (Μηχανικός)	60min	90min	60min



Η διάρκεια της εργασίας A_1, αν ο χρόνος έναρξής της είναι η χρονική στιγμή 2. Στο σχήμα εμφανίζεται το πεδίο της μεταβλητής μετά την εφαρμογή του αντίστοιχου περιορισμού

Παράδειγμα Μετά-Περιορισμών

$A_1 + 4 \leq 7$ δηλαδή $A_1 \leq 3$,

$A_2 + 2 \leq 7$

$B_1 + 1 \leq 7 \wedge B_2 + 3 \leq 7$

$\Gamma_1 + 1 \leq 7 \wedge \Gamma_2 + 2 \leq 7$

Κατά συνέπεια πρέπει:

- είτε να ολοκληρωθεί η A_1 πριν από την A_2, δηλαδή $A_1 + 4 \leq A_2$, ή
- να ολοκληρωθεί η A_2 πριν την A_1, άρα $A_2 + 2 \leq A_1$,

το οποίο μοντελοποιείται με τον μετα-περιορισμό $(A_1 + 4 \leq A_2 \vee A_2 + 2 \leq A_1) = 1$.

Περιορισμοί μη αλληλοεπικάλυψης:

$A_1 + 4 \leq A_2 \vee A_2 + 2 \leq A_1$ (για το αυτοκίνητο Α)

$B_1 + 1 \leq B_2 \vee B_2 + 3 \leq B_1$ (για το αυτοκίνητο Β)

$\Gamma_1 + 1 \leq \Gamma_2 \vee \Gamma_2 + 2 \leq \Gamma_1$ (για το αυτοκίνητο Γ)

Παράδειγμα Μετά-Περιορισμών

Εισάγονται στο σύστημα οι ακόλουθοι περιορισμοί:

$$A_1 + 4 \leq B_1 \vee B_1 + 1 \leq A_1$$
$$A_1 + 4 \leq \Gamma_1 \vee \Gamma_1 + 1 \leq A_1$$
$$B_1 + 1 \leq \Gamma_1 \vee \Gamma_1 + 1 \leq B_1$$

Αντίστοιχα για τον μηχανικό, οι περιορισμοί οι οποίοι εκφράζουν το γεγονός ότι μπορεί να εργαστεί σε ένα αυτοκίνητο κάθε χρονική στιγμή, είναι:

$$A_2 + 2 \leq B_2 \vee B_2 + 3 \leq A_2$$
$$A_2 + 2 \leq \Gamma_2 \vee \Gamma_2 + 2 \leq A_2$$
$$B_2 + 3 \leq \Gamma_2 \vee \Gamma_2 + 2 \leq B_2$$

Παράδειγμα Μετά-Περιορισμών

Η διαδικασία επίλυσης ξεκινά με την εφαρμογή των μοναδιαίων περιορισμών, οπότε τα πεδία των μεταβλητών γίνονται:

$A_1 \in [0..3]$ $A_2 \in [0..5]$
$B_1 \in [1..6]$ $B_2 \in [1..4]$
$\Gamma_1 \in [2..6]$ $\Gamma_2 \in [2..5]$

…τότε ο περιορισμός μη αλληλεπικάλυψης ανάμεσα στις A_1 και A_2, με αντικατάσταση της τιμής της A_2 γίνεται:

$$A_1+4 \leq A_2 \vee A_2+2 \leq A_1 \Rightarrow \underline{A_1+4 \leq 0} \vee 0+2 \leq A_1$$

Η ίδια διαδικασία λαμβάνει χώρα και στους υπόλοιπους περιορισμούς στους οποίους συμμετέχει η A_2:

$$A_2+2 \leq B_2 \vee B_2+3 \leq A_2 \Rightarrow 0+2 \leq B_2 \vee \underline{B_2+3 \leq 0} \Rightarrow 2 \leq B_2$$
$$A_2+2 \leq \Gamma_2 \vee \Gamma_2+2 \leq A_2 \Rightarrow 0+2 \leq \Gamma_2 \vee \underline{\Gamma_2+2 \leq 0} \Rightarrow 2 \leq \Gamma_2$$

Η εφαρμογή των μοναδιαίων περιορισμών που προέκυψαν οδηγούν σε νέα μείωση των πεδίων των μεταβλητών:

$A_1 \in [2..3]$ $A_2 = 0$
$B_1 \in [1..6]$ $B_2 \in [2..4]$
$\Gamma_1 \in [2..6]$ $\Gamma_2 \in [2..5]$

Παράδειγμα Μετά-Περιορισμών

Έστω ότι στο επόμενο βήμα ανατίθεται η τιμή 2 στη μεταβλητή A_1, οπότε:

$$2 + 4 \leq B_1 \vee B_1 + 1 \leq 2 \Rightarrow 6 \leq B_1 \vee B_1 \leq 1$$ (καμία διαγραφή)

$$2 + 4 \leq \Gamma_1 \vee \Gamma_1 + 1 \leq 2 \Rightarrow 6 \leq \Gamma_1 \vee \underline{\Gamma_1 \leq 1} \Rightarrow 6 \leq \Gamma_1$$

…Όμως:

$$\Gamma_1 + 1 \leq \Gamma_2 \vee \Gamma_2 + 2 \leq \Gamma_1 \Rightarrow \underline{6 + 1 \leq \Gamma_2} \vee \Gamma_2 + 2 \leq 6 \Rightarrow \Gamma_2 \leq 4$$

εφόσον το $\Gamma_2 \in [2..5]$ και επίσης

$$B_1 + 1 \leq \Gamma_1 \vee \Gamma_1 + 1 \leq B_1 \Rightarrow B_1 + 1 \leq 6 \vee 6 + 1 \leq B_1 \Rightarrow$$

$$\Rightarrow B_1 \leq 5 \vee \underline{7 \leq B_1} \Rightarrow B_1 \leq 5$$

Τα νέα πεδία είναι:

$A_1 \in 2$	$A_2 = 0$
$B_1 \in [1..5]$	$B_2 \in [2..4]$
$\Gamma_1 \in 6$	$\Gamma_2 \in [2..4]$

… η διαδικασία καταλήγει στη λύση:

$$A_1 = 2, A_2 = 0, B_1 = 1, B_2 = 4, \Gamma_1 = 6, \Gamma_2 = 2$$

Πρόβλημα

Το πρόβλημα των κανιβάλων και των ιεραποστόλων υπάρχουν αρχικά τρεις κανίβαλοι και τρεις ιεραπόστολοι στην αριστερή όχθη του ποταμού. Στόχος μας είναι να μεταφέρουμε με τη βάρκα όλους τους ανθρώπους στη δεξιά όχθη. Η βάρκα χωρά το πολύ δύο άτομα και δεν μπορεί να μετακινηθεί χωρίς επιβάτες. Δεν πρέπει ποτέ ο αριθμός των κανιβάλων σε κάποια όχθη να υπερβεί τον αριθμό των ιεραποστόλων στην ίδια όχθη. Οι δυνατές κινήσεις σε κάθε κατάσταση είναι η μετακίνηση ενός ή δύο ανθρώπων με τη βάρκα στην απέναντι όχθη.

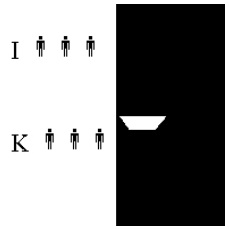
Λύση είναι κάθε ακολουθία κινήσεων που πετυχαίνει τον επιθυμητό στόχο. Το κόστος μιας λύσης είναι ο συνολικός αριθμός μετακινήσεων της βάρκας.

(α) Χρησιμοποιώντας την τεχνική αφαίρεσης περιορισμών, κατασκευάστε δύο αποδεκτές ευριστικές συναρτήσεις *h1* και *h2*, ώστε η *h1* να κυριαρχεί επί της *h2*. Γράψτε τους τύπους των *h1(s)* και *h2(s)*, όπου *s* η κατάσταση του χώρου αναζήτησης που αξιολογείται, και εξηγήστε πώς προκύπτουν οι τύποι και γιατί η *h1(s)* κυριαρχεί επί της *h2(s)*. Μπορείτε να καλέσετε *n* το συνολικό αριθμό ανθρώπων που βρίσκονται στην αριστερή όχθη στην κατάσταση *s*. Υπόδειξη: Σε μία κατάσταση η βάρκα μπορεί να βρίσκεται είτε στην αριστερή είτε στη δεξιά όχθη. Ίσως χρειαστεί να δώσετε διαφορετικούς τύπους για κάθε μία από τις δύο περιπτώσεις.

- *h1(s)* και πώς προκύπτει:
- *h2(s)* και πώς προκύπτει:
- Η *h1(s)* κυριαρχεί της *h2(s)* επειδή …….

Απάντηση (α)

- Αφαιρούμε τον περιορισμό της αριθμητικής σχέσης κανιβάλων – ιεραποστόλων.
  - $h_1(s)=2(n-1)-1$, όταν η βάρκα είναι αριστερά και $n>1$
  - $h_1(s)=1$, όταν η βάρκα είναι αριστερά και $n=1$
  - $h_1(s)=2n$, όταν η βάρκα είναι δεξιά

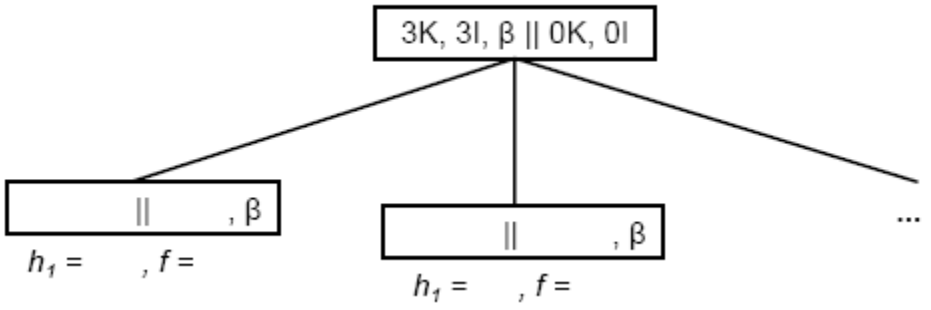


- Αφαιρώντας τον περιορισμό της χωρητικότητας της βάρκας.
  - $h_2(s)=1$, όταν η βάρκα είναι αριστερά
  - $h_2(s)=2$, όταν η βάρκα είναι δεξιά

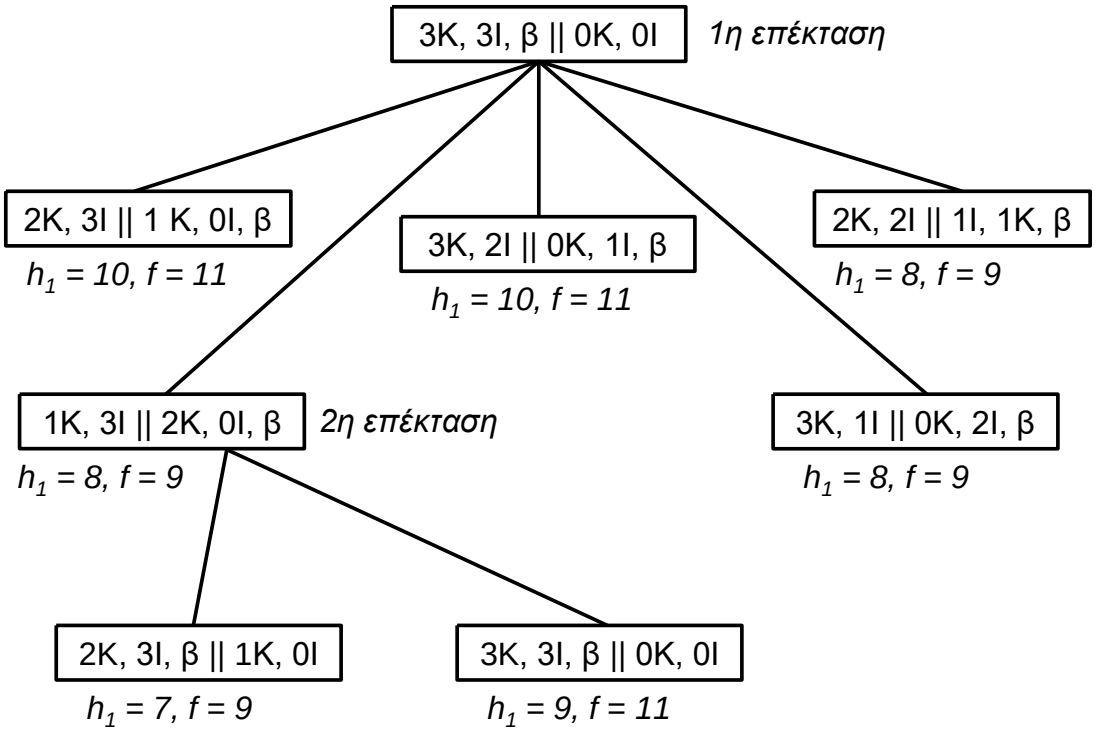
- Η h_1 κυριαρχεί της h_2 γιατί για κάθε s έχουμε $h_2(s) \leq h_1(s)$

Πρόβλημα

Χρησιμοποιούμε τον αλγόριθμο A* με ευριστική συνάρτηση την *h1*. Συμπληρώστε το παρακάτω δέντρο αναζήτησης με τα παιδιά της ρίζας. Κυκλώστε το παιδί της ρίζας που αποφασίζει να επεκτείνει πρώτα ο A*. Συμπληρώστε στο δέντρο τους κόμβους που προκύπτουν από αυτή την επέκταση. Κυκλώστε τον κόμβο που αποφασίζει να επεκτείνει κατόπιν ο A* (δεν χρειάζεται να συμπληρώσετε τα νέα παιδιά, αν δεν υπάρχουν ήδη στο διάγραμμα). Κάτω από κάθε κόμβο να φαίνονται τα μέτρα που χρησιμοποιεί ο A* για να αποφασίσει ποιον κόμβο θα επεκτείνει στη συνέχεια.



Απάντηση (β)


3K, 3I, β || 0K, 0I
1η επέκταση
2K, 3I || 1 K, 0I, β
h1 = 10, f = 11
3K, 2I || 0K, 1I, β
h1 = 10, f = 11
2K, 2I || 1I, 1K, β
h1 = 8, f = 9
1K, 3I || 2K, 0I, β
2η επέκταση
h1 = 8, f = 9
3K, 1I || 0K, 2I, β
h1 = 8, f = 9
2K, 3I, β || 1K, 0I
h1 = 7, f = 9
3K, 3I, β || 0K, 0I
h1 = 9, f = 11

Πρόβλημα

(γ) Γιατί οι ευριστικές συναρτήσεις που προκύπτουν με αφαίρεση περιορισμών είναι αποδεκτές;

(δ) Θα βρει ο Α* πρώτα τη βέλτιστη λύση αν χρησιμοποιήσουμε οποιαδήποτε από τις *h1* και *h2*; Γιατί;

(ε) Υπενθυμίζεται ότι αν το κόστος κάθε μετάβασης μεταξύ καταστάσεων είναι θετικό, οι ευριστικές συναρτήσεις που προκύπτουν με αφαίρεση περιορισμών είναι και συνεπείς. Τι μας προσφέρει η χρήση συνεπούς ευριστικής συνάρτησης όταν χρησιμοποιούμε κλειστό σύνολο με τον Α*;

(στ) Θεωρήστε την περίπτωση όπου το κόστος κάθε μεταβάσεως μεταξύ καταστάσεων είναι θετικός αριθμός. Αν ένα πρόβλημα έχει λύση και καμία κατάσταση δεν έχει άπειρα παιδιά, είναι δυνατόν ο Α* να μην βρει τη λύση;

Απάντηση (γ,δ,ε,στ)

γ) Γιατί αποτελούν το ακριβές βέλτιστο κόστος όταν αγνοούνται κάποιοι περιορισμοί του αρχικού προβλήματος.

δ) Ναι, γιατί ο A* είναι βέλτιστος όταν χρησιμοποιείται αποδεκτή ευριστική.

ε) Με συνεπή h, ο A* παραμένει βέλτιστος και όταν χρησιμοποιούμε κλειστό σύνολο.

στ) Όχι γιατί A* είναι πλήρης αν το b είναι πεπερασμένο και το κόστος κάθε μετάβασης είναι >0.

Αναφορές

- Τεχνητή Νοημοσύνη Δ’ Έκδοση Ι. Βλαχάβας, Π. Κεφαλάς, Ν. Βασιλειάδης, Φ. Κόκκορας, Η. Σακελλαρίου, https://aibook.gr/
- Τεχνητή Νοημοσύνη – Μια σύγχρονη προσέγγιση, Β’ έκδοση, Stuart Russell, Peter Norvig, Εκδόσεις Κλειδάριθμος, 2003.
- http://aispace.org/constraint/